

Hoare Logic

LN chapter 5, 6.1, 6.2

Hoare Logic is used to reason about the correctness of programs. In the end, it reduces a program and its specification to a set of verification conditions.

Overview

- Hoare logic (Ch 5, 6.1)
 - Hoare triple
 - Rules for basic statements
 - Weakest pre-condition
 - Example
- Weakest pre-condition (LN 6.2)

// SEQ, IF, ASG

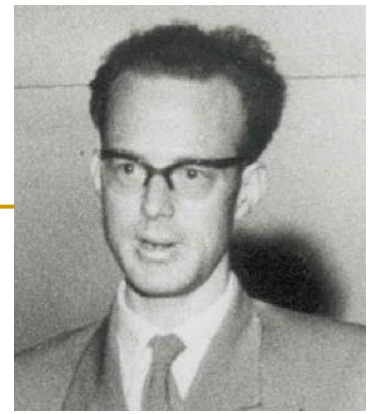
Hoare logic basics

LN Chapter 5, 6.1

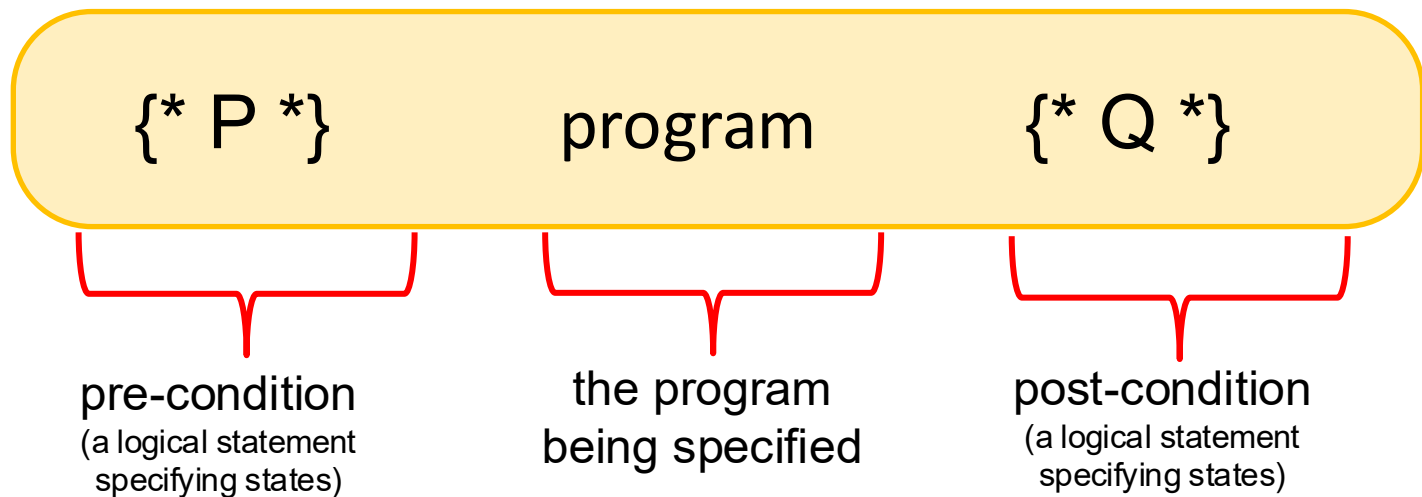
Why Hoare logic?

- We've looked at proofs about **purely functional** programs
 - e.g., `rev` and `rv` are purely functional
 - $\forall s :: \text{rev } s = \text{rv } [] \ s$
 - **Purely functional** programs only take in an input and return an output; the behavior is not affected by anything else and it does not affect anything else
 - Thus, we prove properties about it mathematically, as above
- Now we reason about **stateful**/imperative/object oriented programs
 - These are affected by and affect the state of the memory, etc.

Hoare logic



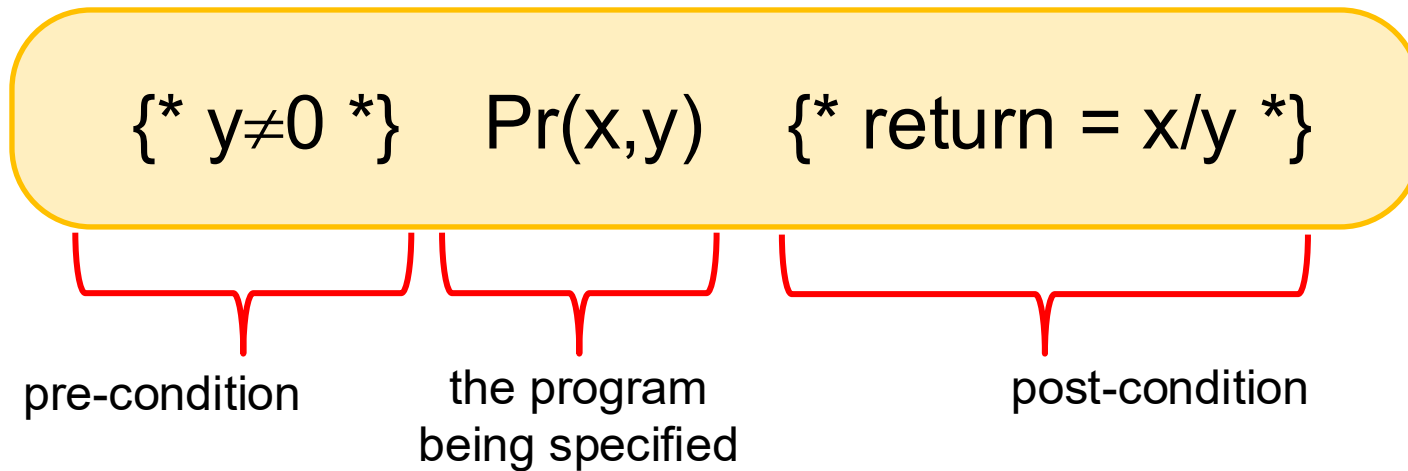
- Invented by Tony Hoare (1934-2026) in 1969
 - Influenced by Robert Floyd
- The **Hoare triple** expresses the effect the program has on state



- The triple says: if we are in a state satisfying P , after the program runs, we are in a state satisfying Q .

Hoare triple

- Example:



- Meaning: if we are in a state where $y \neq 0$, then after running $\text{Pr}(x, y)$, we are in a state where $\text{return} = x/y$.

Partial and total correctness interpretation

$$\{^* P ^*\} \quad \text{Pr}(x) \quad \{^* Q ^*\}$$

- **Total correctness interpretation** of Hoare triple: if the program is executed on a state satisfying P , it will **terminate** in a state satisfying Q .
- **Partial correctness interpretation**: if the program is executed on a state satisfying P , and **if it terminates**, it will terminate in a state satisfying Q .
- Partial correctness is of course weaker, but on the other hand easier to prove. Useful in situations where we can afford to postpone concerns about termination.

Limitations of Hoare logic

- Not every type of behavioral property can (naturally) be captured with Hoare logic.
- Examples:
 - If we want to reason about the order in which certain things happen when a program is run.
 - Alternative: CSP (communicating sequential process)
 - If we want to reason about how often a state is visited.
 - Alternative: temporal logic
 - If we want to reason about trusting information.
 - Alternative: logic of belief.

Hoare Logic

Components

- Basic statement is the Hoare triple
 - $\{*P*\} \text{ program } \{*Q*\}$
- To prove a specific statement, we use **inference rules**, like the one below
 - It says that if the statements above the line hold, then we can conclude the statements below the line

$$\frac{\{*P*\} \text{ S } \{*Q*\} \qquad \{*Q*\} \text{ T } \{*R*\}}{\{*P*\} \text{ S ; T } \{*R*\}}$$

Sequential decomposition

- The first rule is the most important

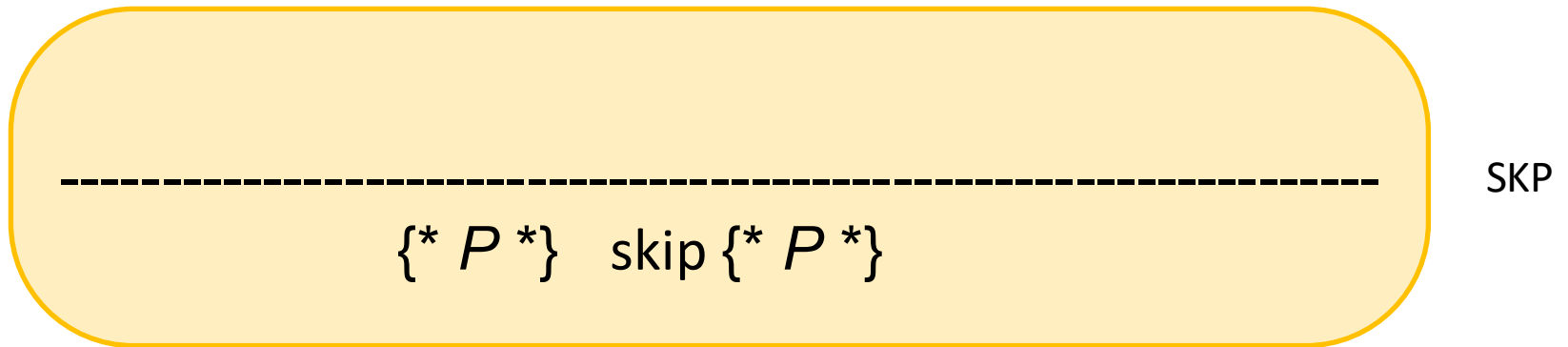
$$\frac{\{ * P * \} \quad S \quad \{ * Q * \} \qquad \{ * Q * \} \quad T \quad \{ * R * \}}{\{ * P * \} \quad S ; T \quad \{ * R * \}}$$

SEQ
(B.1.6)

- It tells us that if we want to reason about a program consisting of two statements S and T in sequence, then we can do this by reasoning about the programs S and T separately
- Thus, we break down a program into its steps and reason one-by-one

Skip

- Here is another basic rule



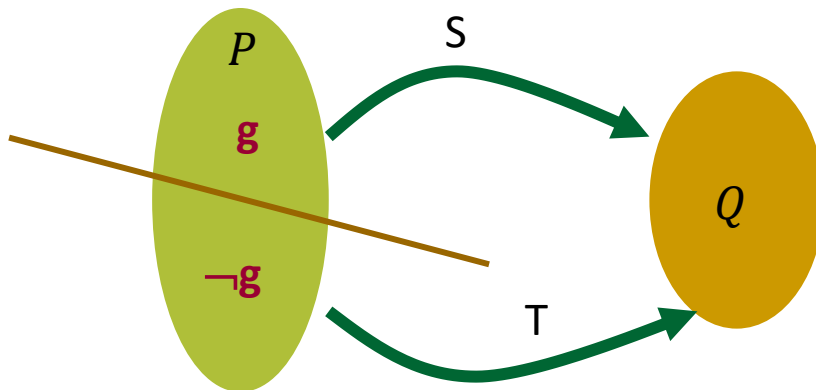
- If P holds before executing skip, then it still holds afterwards

Inference rule for if

$$\{^* P \wedge g ^*\} \text{ S } \{^* Q ^*\}$$
$$\{^* P \wedge \neg g ^*\} \text{ T } \{^* Q ^*\}$$

$$\{^* P ^*\} \text{ if } g \text{ then S else T } \{^* Q ^*\}$$

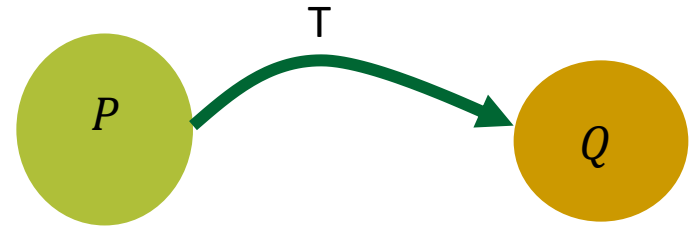
IF
(B.1.5)

$$\{^* P \wedge g ^*\} \text{ S } \{^* Q ^*\}$$

$$\{^* P \wedge \neg g ^*\} \text{ T } \{^* Q ^*\}$$

Mental model:

- Sometimes think of P as a **logical statement**
- Sometimes think of P as the **set of all states satisfying P**

Predicates as subsets



- Let W denote the set of all possible states.
- Let $|P|$ denote $\{s \in W \mid P s\}$, that is the set of all states where P holds.
 - This gives a bijection between subsets $|P|$ and predicates P , meaning that we are justified in switching back and forth between the mental representations.
 - We usually let P denote $|P|$, unless it's confusing.
- If our program is deterministic, a program T gives rise to a function

$$T : W \rightarrow W$$

- When we draw a picture as above, we illustrate the fact that

$$\{* P *\} \xrightarrow{T} \{* Q *\}$$

or, equivalently, that

$$T(P) \subseteq Q$$

i.e., that T takes the states satisfying P into the states satisfying Q .

Inference rule for assignment

??

$\{^* P ^*\} \quad x:=e \quad \{^* Q ^*\}$

- **Idea:** let's first construct a pre-condition W that holds before the assignment **if and only if** Q holds after the assignment.
- Then we can equivalently prove $P \Rightarrow W$

Examples

- $\{^* W? ^*\} \text{ } x:=10 \{^* x = y ^*\}$
 $W: 10 = y$
- $\{^* W? ^*\} \text{ } x:=x+a \{^* x = y ^*\}$
 $W: x + a = y$
- $\{^* W? ^*\} \text{ } x:=x+1 \{^* x > 10 ^*\}$
 $W: x + 1 > 10$
- Generally: for $\{^*P^*\} x := e \{^*Q^*\}$, set $W := Q[e/x]$
 - $Q[e/x]$ is the statement obtained from Q by replacing all free occurrences of x by e

Assignment

- Theorem:

*Q holds after $\mathbf{x}:=\mathbf{e}$
if and only if
 $Q[e/x]$ held before $\mathbf{x}:=\mathbf{e}$.*

$$\{^* Q[e/x] ^*\} \mathbf{x}:=\mathbf{e} \{^* Q ^*\}$$

ASG

More basic rules

■ Pre-condition strengthening:

$$\vdash P \Rightarrow Q \quad , \quad \{ * Q * \} \quad S \quad \{ * R * \}$$

$$\{ * P * \} \quad S \quad \{ * R * \}$$

PRE
(B.1.2)

■ Post-condition weakening:

$$\{ * P * \} \quad S \quad \{ * Q * \} \quad , \quad \vdash Q \Rightarrow R$$

$$\{ * P * \} \quad S \quad \{ * R * \}$$

POST
(B.1.1)

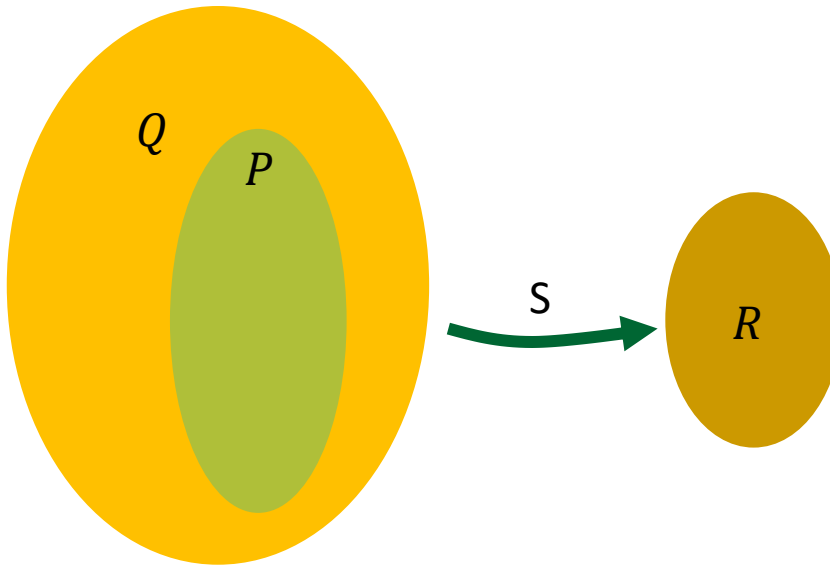
Pre-condition strengthening picture

■ Pre-condition strengthening:

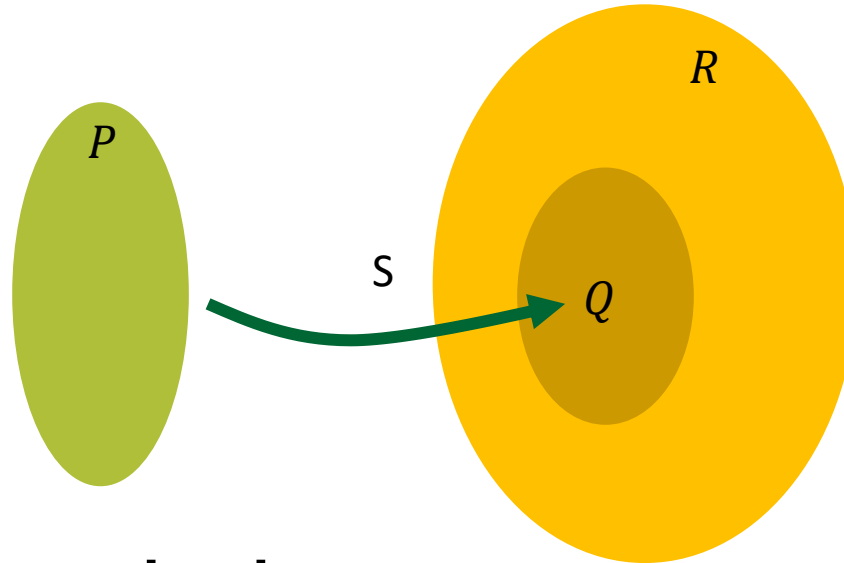
$$\vdash P \Rightarrow Q \quad , \quad \{^* Q ^*\} S \{^* R ^*\}$$

$$\{^* P ^*\} S \{^* R ^*\}$$

PRE
(B.1.2)



Post-condition weakening picture



■ Post-condition weakening:

$$\{^* P ^*\} \ S \ \{^* Q ^*\} \ , \ \vdash Q \Rightarrow R$$

$$\{^* P ^*\} \ S \ \{^* R ^*\}$$

POST
(B.1.1)

PRE + ASG

$$\frac{\begin{array}{c} \text{----- ASG} \\ \vdash P \Rightarrow Q[e/x] \quad \{^* Q[e/x] ^*\} \mathbf{x} := \mathbf{e} \{^* Q ^*\} \\ \text{----- PRE} \end{array}}{\{^* P ^*\} \mathbf{x} := \mathbf{e} \{^* Q ^*\}}$$

is a proof of

$$\frac{\vdash P \Rightarrow Q[e/x]}{\{^* P ^*\} \mathbf{x} := \mathbf{e} \{^* Q ^*\}}$$

More basic rules

■ Conjunction of Hoare triples:

$$\frac{\{^* P_1 ^*\} \text{ S } \{^* Q_1 ^*\} \quad \{^* P_2 ^*\} \text{ S } \{^* Q_2 ^*\}}{\{^* P_1 \wedge P_2 ^*\} \text{ S } \{^* Q_1 \wedge Q_2 ^*\}}$$

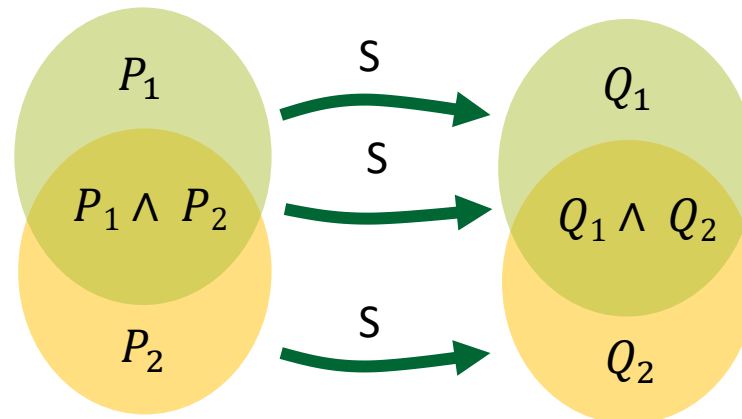
■ Disjunction of Hoare triples

$$\frac{\{^* P_1 ^*\} \text{ S } \{^* Q_1 ^*\} \quad \{^* P_2 ^*\} \text{ S } \{^* Q_2 ^*\}}{\{^* P_1 \vee P_2 ^*\} \text{ S } \{^* Q_1 \vee Q_2 ^*\}}$$

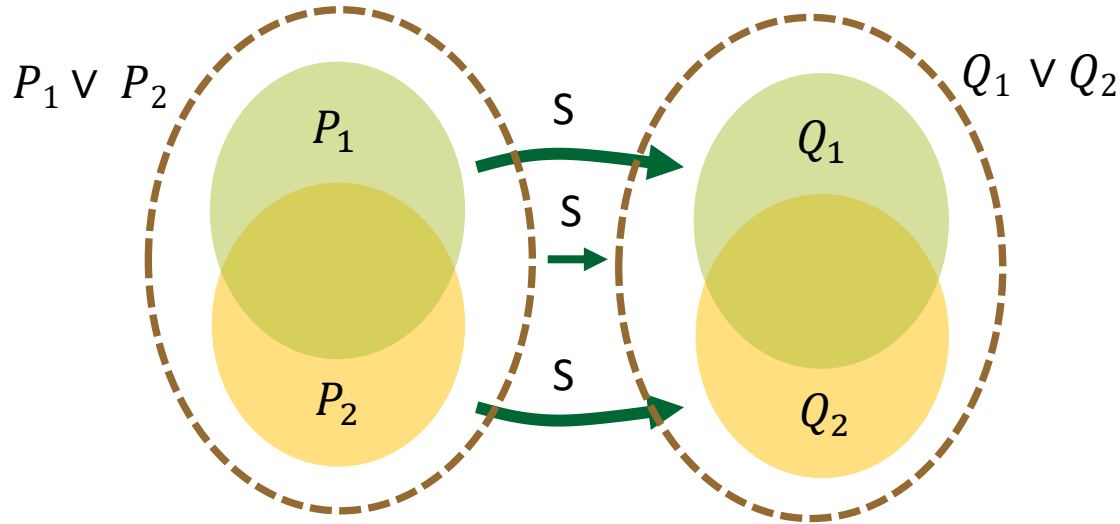
Conjunction picture

■ Conjunction of Hoare triples:

$$\frac{\{^* P_1 ^*\} \text{ S } \{^* Q_1 ^*\} \quad \{^* P_2 ^*\} \text{ S } \{^* Q_2 ^*\}}{\{^* P_1 \wedge P_2 ^*\} \text{ S } \{^* Q_1 \wedge Q_2 ^*\}}$$



Disjunction picture



■ Disjunction of Hoare triples

$$\begin{array}{c} \{^* P_1 ^*\} \quad S \quad \{^* Q_1 ^*\} \qquad \{^* P_2 ^*\} \quad S \quad \{^* Q_2 ^*\} \\ \hline \{^* P_1 \vee P_2 ^*\} \quad S \quad \{^* Q_1 \vee Q_2 ^*\} \end{array}$$

Weakest pre-condition

LN Section 6.2

How does a proof proceed now ?

- Example:

$\{^* x \neq y \ ^*\} \quad \text{tmp} := x ; x := y ; y := \text{tmp} \quad \{^* x \neq y \ ^*\}$

- To use the SEQ rule you need intermediate assertions:

$\{^* x \neq y \ ^*\} \quad \text{tmp} := x \quad \{^* ? \ ^*\} ; x := y \quad \{^* ? \ ^*\} ; y := \text{tmp} \quad \{^* x \neq y \ ^*\}$

- Inference rules of Hoare logic tell us how to prove the claim once we have intermediate assertions
- But they **do not** tell us how to come up with these intermediate assertions in the first place.

Weakest pre-condition

- We want a function

$wp : \text{Stmt} \rightarrow \text{Pred} \rightarrow \text{Pred}$

- that takes a program statement and a predicate on states
- and returns a predicate on states
such that
- $\{^* wp\ S\ Q\ ^*\} S \{^* Q\ ^*\}$
- and $wp\ S\ Q$ is the **weakest** pre-condition making the above hold.
- Then if $\{^* P\ ^*\} S \{^* Q\ ^*\}$ held, we would get $P \Rightarrow wp\ S\ Q$.

Weakest pre-condition

- So we characterize **wp** as follows (Def. 6.2.3):

$$\{ * P * \} S \{ * Q * \} \quad \equiv \quad P \Rightarrow \mathbf{wp} S Q$$

- *Corollary-1*: Such a **wp** is a valid pre-condition. That is, you can prove:

$$\{ * \mathbf{wp} S Q * \} S \{ * Q * \}$$

- *Corollary-2* : **wp** reduces program verification problems to proving implications, for which we already have a “tool” for (Predicate Logic).

Weakest pre-condition

- Again, the characterization of wp:

$$\{^* P ^*\} \text{ S } \{^* Q ^*\} \quad \equiv \quad P \Rightarrow \text{wp S } Q$$

- *Corollary-3*: the reduction is complete. That is, if either side above is not valid, neither is the other side.
- In particular: a counterexample invalidating $P \Rightarrow \text{wp S } Q$ is a state satisfying P that after running S produces a state that is not in Q . Such an input is useful for debugging S .

But...

- But this characterization is **not constructive**:

$$\{^* P ^*\} S \{^* Q ^*\} \equiv P \Rightarrow \text{wp } S Q$$

- That is, it does not tell us how to actually calculate **wp** $S Q$.
- To be actually usable, we need to come up with a constructive definition for wp.

Some notes before we attempt to define wp

- This is the meta property we want to have:

$$P \Rightarrow \text{wp } S \ Q \quad \equiv \quad \{^* P ^*\} \ S \ \{^* Q ^*\}$$

- A definition/implementation of **wp** should be at least “**sound**”:

$$P \Rightarrow \text{wp } S \ Q \quad \Rightarrow \quad \{^* P ^*\} \ S \ \{^* Q ^*\}$$

- It should also be “**complete**”:

$$P \Rightarrow \text{wp } S \ Q \quad \Leftarrow \quad \{^* P ^*\} \ S \ \{^* Q ^*\}$$

Or, Equivalently:

$$\cancel{P \Rightarrow \text{wp } S \ Q} \quad \Rightarrow \quad \cancel{\{^* P ^*\} \ S \ \{^* Q ^*\}}$$

Subset view of wp

- This is the meta property we want to have:

$$P \Rightarrow \text{wp } S Q \equiv \{^* P ^*\} S \{^* Q ^*\}$$

- In terms of subsets, this is equivalently stated as:

$$P \subseteq \text{wp } S Q \equiv S(P) \subseteq Q$$

But notice that the following is true (from logic / basic set theory)

$$P \subseteq S^{-1}(Q) \equiv S(P) \subseteq Q$$

Therefore, $S^{-1}(Q)$ has the property we want!

$$\text{wp } S Q = S^{-1}(Q)$$

- It also matches our intuition: $\text{wp } S Q$ is the collection of all states that are sent to Q by S .

Subset view of wp

- Let's check that if we have

$$(1) \quad P \subseteq \text{wp} \, S \, Q \quad \equiv \quad S(P) \subseteq Q \quad (\text{assumption about wp})$$

and

$$(2) \quad P \subseteq S^{-1}(Q) \quad \equiv \quad S(P) \subseteq Q \quad (\text{true})$$

then $\text{wp} \, S \, Q = S^{-1}(Q)$.

Proof. Putting (1) and (2) together, we get

$$P \subseteq \text{wp} \, S \, Q \quad \equiv \quad P \subseteq S^{-1}(Q).$$

Since $\text{wp} \, S \, Q \subseteq \text{wp} \, S \, Q$ we get $\text{wp} \, S \, Q \subseteq S^{-1}(Q)$.

Similarly, since $S^{-1}(Q) \subseteq S^{-1}(Q)$, we get $S^{-1}(Q) \subseteq \text{wp} \, S \, Q$.

Thus, $\text{wp} \, S \, Q = S^{-1}(Q)$.

Deterministic vs. non-deterministic

- Remember we are assuming that our programs are deterministic. That is, we model a program P as a function

$$P: \text{States} \rightarrow \text{States}.$$

- Programs can also be nondeterministic. Then we model a program P as a function

$$P: \text{States} \rightarrow \mathcal{P}(\text{States})$$

- Here $\mathcal{P}(\text{States})$ is the **Powerset** of States. That means its elements are subsets of States.
 - Thus, a nondeterministic program P is one that takes a single state s to a set $P(s)$ of *possible states*.
- Proofs of our statements are only slightly more complicated for nondeterministic programs. See the lecture notes.

Weakest pre-condition of skip

$$\text{wp} \quad \text{skip} \quad Q \quad = \quad Q$$

Proof: As a function, **skip** is the identity.
Therefore,

$$\begin{aligned} & \text{skip}^{-1} (Q) \\ &= id^{-1} \\ &= Q. \end{aligned}$$

Inference rule for skip

$$\text{wp skip } Q = Q$$

From the above, we get the following inference rule (if we didn't know it before).

$$\frac{}{\{^* Q *\} \text{ skip } \{^* Q *\}}$$

SKP

Weakest pre-condition of assignment

$$\text{wp } (\mathbf{x}:=\mathbf{e}) \ Q = Q[\mathbf{e}/\mathbf{x}]$$

We already showed this! (slide 16)

Proof (in terms the preimage): We want to show $(\mathbf{x}:=\mathbf{e})^{-1} (Q) = Q[\mathbf{e}/\mathbf{x}]$. For any state s , we have

$$\begin{aligned} & s \in (\mathbf{x}:=\mathbf{e})^{-1} (Q) \\ = & (\mathbf{x}:=\mathbf{e}) \ s \in Q \\ = & (\mathbf{x}:=\mathbf{e}) \ s \text{ satisfies } Q \\ = & s \text{ satisfies } Q[\mathbf{e}/\mathbf{x}] \\ = & s \in Q[\mathbf{e}/\mathbf{x}]. \end{aligned}$$

Inference rule for assignment

- Now, if we didn't already know the inference rule for assignment, we would be able to assert

$$\frac{}{\{^* Q[e/x] ^*\} \mathbf{x}:=\mathbf{e} \{^* Q ^*\}}$$

ASG

or, equivalently:

$$\vdash P \Rightarrow Q[e/x]$$

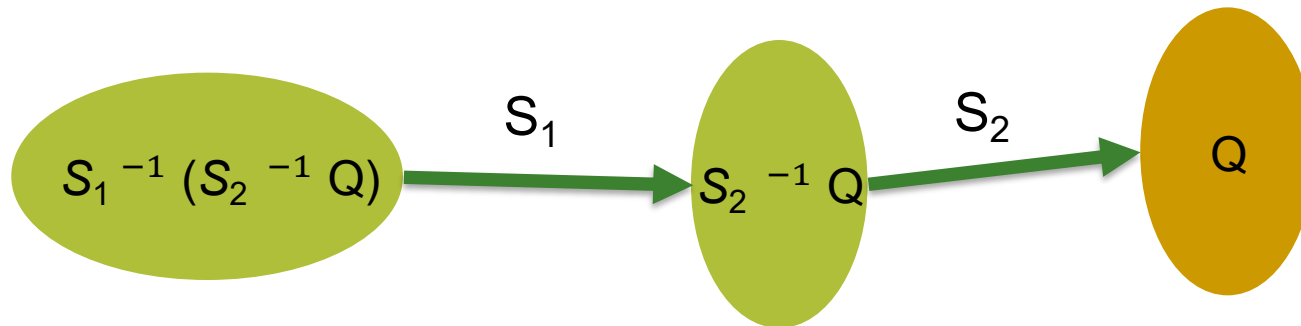
$$\frac{}{\{^* P ^*\} \mathbf{x}:=\mathbf{e} \{^* Q ^*\}}$$

wp of SEQ

$$\text{wp } (S_1 ; S_2) \ Q = \text{wp } S_1 \ (\text{wp } S_2 \ Q)$$

Proof: We have that

$$\begin{aligned} & \text{wp } (S_1 ; S_2) \ Q \\ &= (S_1 ; S_2)^{-1} Q \\ &= (S_2 \circ S_1)^{-1} Q \\ &= S_1^{-1} (S_2^{-1} Q) \\ &= \text{wp } S_1 \ (\text{wp } S_2 \ Q). \end{aligned}$$



Post condition weakening

$$(Q \Rightarrow Q') \quad \Rightarrow \quad (\text{wp } S \ Q \Rightarrow \text{wp } S \ Q')$$

Proof: We want to show

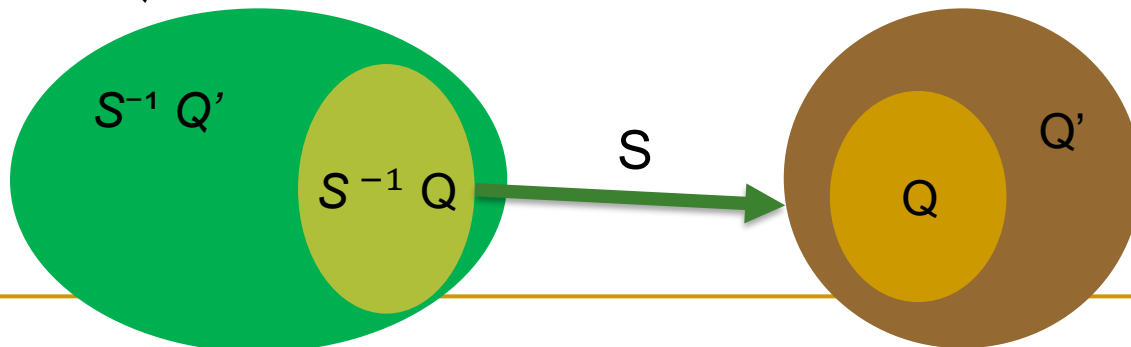
$$Q \subseteq Q' \quad \Rightarrow \quad \text{wp } S \ Q \subseteq \text{wp } S \ Q'$$

or equivalently,

$$Q \subseteq Q' \quad \Rightarrow \quad S^{-1} Q \subseteq S^{-1} Q'.$$

Suppose $Q \subseteq Q'$. Consider $s \in S^{-1} Q$ with the goal of showing $s \in S^{-1} Q'$.

$$\begin{aligned} & s \in S^{-1} Q \\ \Rightarrow & S s \in Q \\ \Rightarrow & S s \in Q'. \\ \Rightarrow & s \in S^{-1} Q'. \end{aligned}$$



SEQ and weakening

Recall:

$$\begin{array}{lcl} \text{wp } (S_1 ; S_2) R & = & \text{wp } S_1 (\text{wp } S_2 R) \\ (Q \Rightarrow Q') & \Rightarrow & (\text{wp } S Q \Rightarrow \text{wp } S Q') \end{array}$$

Pretend we don't already know the inference rule below.

Consider P, S, Q, T, R satisfying its hypotheses.

Then $Q \Rightarrow \text{wp } S_2 R$ so $\text{wp } S_1 Q \Rightarrow \text{wp } S_1 (\text{wp } S_2 R)$ (by weakening).

But $P \Rightarrow \text{wp } S_1 Q \Rightarrow \text{wp } S_1 (\text{wp } S_2 R) = \text{wp } (S_1 ; S_2) R$.

Therefore, if we didn't already know the inference rule SEQ, we could conclude

$$\frac{\{ * P * \} S \{ * Q * \} \quad \{ * Q * \} T \{ * R * \}}{\{ * P * \} S ; T \{ * R * \}}$$

SEQ
(B.1.6)

wp of IF

$$\mathbf{wp} \text{ (if } g \text{ then } S_1 \text{ else } S_2) Q = (g \wedge \mathbf{wp} S_1 Q) \vee (\neg g \wedge \mathbf{wp} S_2 Q)$$

Proof: As a function, **if** g **then** S_1 **else** S_2 does the following to a state s :

$$(\text{if } g \text{ then } S_1 \text{ else } S_2) s = \begin{cases} S_1(s) & \text{if } s \in g \\ S_2(s) & \text{o/w} \end{cases}$$

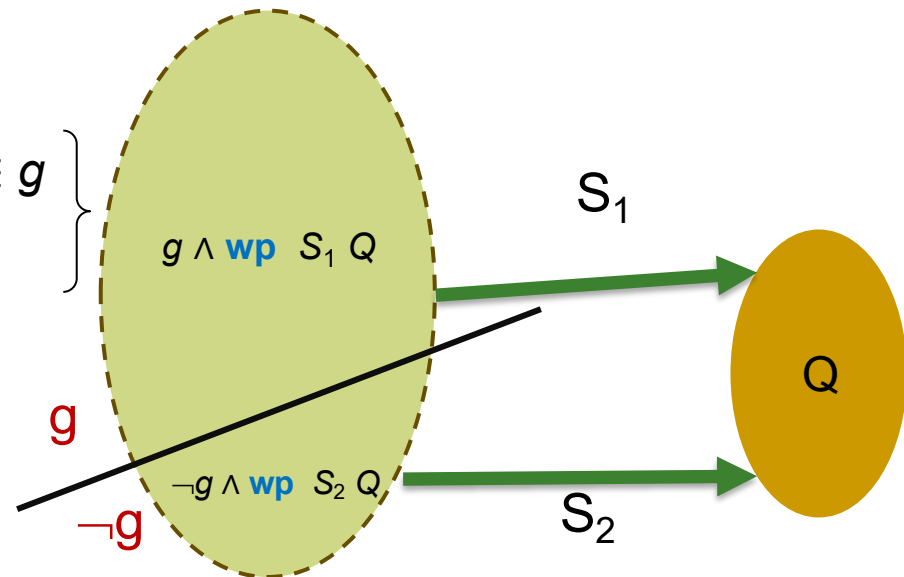
So

$$(\text{if } g \text{ then } S_1 \text{ else } S_2)^{-1} Q$$

$$= \left\{ \begin{array}{ll} S_1 & \text{on } g \\ S_2 & \text{on } \bar{g} \end{array} \right\}^{-1} Q$$

$$= (g \cap S_1^{-1} Q) \cup (\bar{g} \cap S_2^{-1} Q)$$

$$= (g \wedge \mathbf{wp} S_1 Q) \vee (\neg g \wedge \mathbf{wp} S_2 Q).$$



Alternative formulation of wp IF

$$\mathbf{wp} \text{ (if } g \text{ then } S_1 \text{ else } S_2) Q = (g \wedge \mathbf{wp} S_1 Q) \vee (\neg g \wedge \mathbf{wp} S_2 Q)$$

=

$$(g \Rightarrow \mathbf{wp} S_1 Q) \wedge (\neg g \Rightarrow \mathbf{wp} S_2 Q)$$

This formulation for the **wp** of if-then-else is more convenient for working out proofs. E.g. in a deductive proof its conjunctive form would translate to proving two goals.

Inference rule for IF

$$\mathbf{wp} \text{ (if } g \text{ then } S_1 \text{ else } S_2) Q = (g \wedge \mathbf{wp} S_1 Q) \vee (\neg g \wedge \mathbf{wp} S_2 Q)$$

Consider the hypotheses of the inference rule below. Then

$$P \wedge g \Rightarrow \mathbf{wp} S Q \text{ and } P \wedge \neg g \Rightarrow \mathbf{wp} T Q.$$

We have

$$\begin{aligned} P &= P \wedge (g \vee \neg g) && \text{(law of excluded middle)} \\ &= (P \wedge g \wedge g) \vee (P \wedge \neg g \wedge \neg g) \\ &\Rightarrow ((\mathbf{wp} S Q) \wedge g) \vee (\mathbf{wp} T Q \wedge \neg g) \end{aligned}$$

So we get the rule again:

$$\frac{\{* P \wedge g *\} S \{* Q *\} \quad \{* P \wedge \neg g *\} T \{* Q *\}}{\{* P *\} \text{ if } g \text{ then } S \text{ else } T \{* Q *\}}$$

IF
(B.1.5)

How does a proof proceed now ?

- $\{^* x \neq y \ ^*\} \quad \text{tmp} := x \ ; \ x := y \ ; \ y := \text{tmp} \quad \{^* x \neq y \ ^*\}$

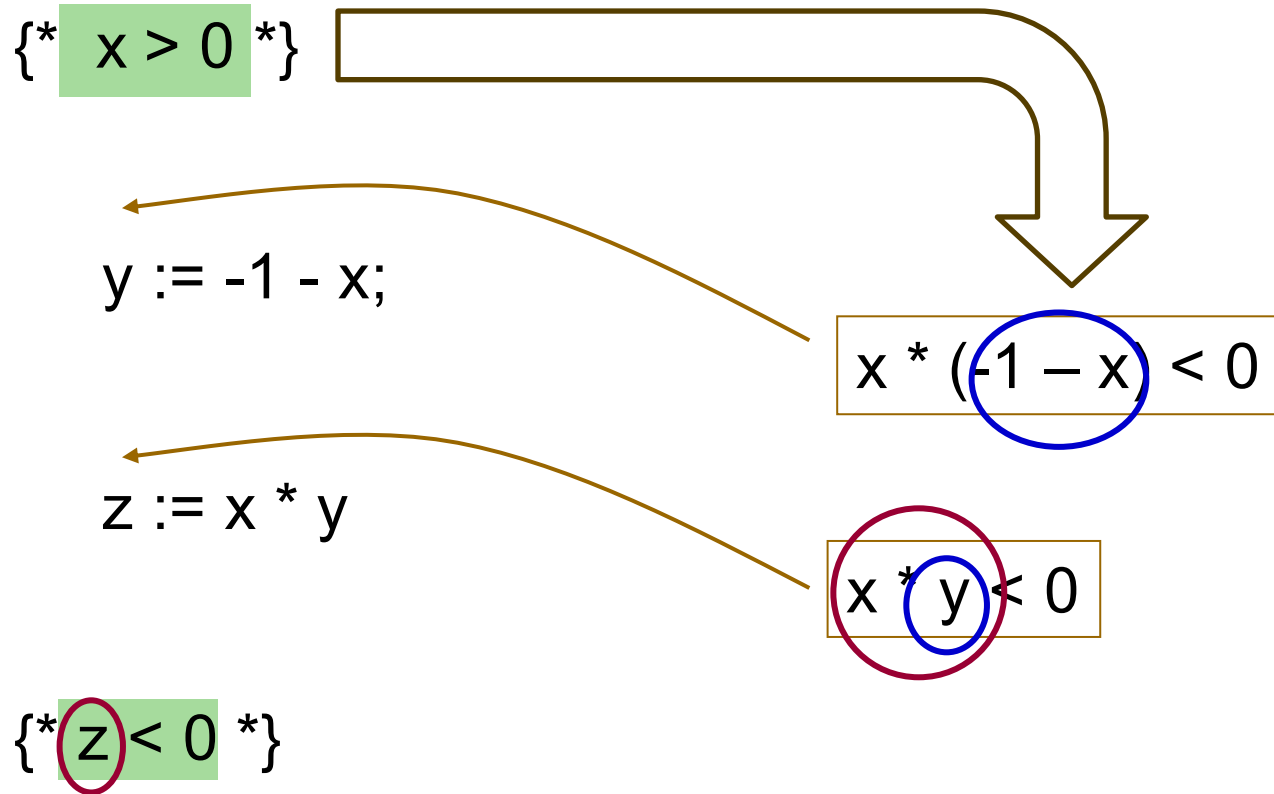
1. Calculate:

$$W \quad = \quad \text{wp} \quad (\text{tmp} := x \ ; \ x := y \ ; \ y := \text{tmp}) \quad x \neq y$$

2. Then prove: $x \neq y \Rightarrow W$

- We calculate the intermediate assertions, rather than manually figuring them out!

Example 1



$$\text{wp } (x := e) \ Q = Q[e/x]$$

Example 1

Thm. $x > 0$ implies $x * (-1 - x) < 0$.

Proof. $x > 0$

$$\Rightarrow 1 + x > 0$$

$$\Rightarrow -(1 + x) < 0$$

$$\Rightarrow -1 - x < 0.$$

Since $x > 0$ and $-1 - x < 0$, the product $x * (-1 - x) < 0$.

Example 1

PROOF top

[A:] $x > 0$

[G:] $\text{wp } (y := -1-x; z := x*y) (z < 0)$

BEGIN-----

1. $\{\text{sub}\} (\text{wp } (y := -1-x; z := x*y) (z < 0)) = (x * (-1 - x) < 0)$

EQUATIONAL PROOF sub -----

$\text{wp } (y := -1-x; z := x*y) (z < 0)$

$= \{\text{wp formula for sequence}\}$

$\text{wp } (y := -1-x) (\text{wp } (z := x*y) (z < 0))$

$= \{\text{wp formula for assignment}\}$

$\text{wp } (y := -1-x) (x*y < 0)$

$= \{\text{wp formula for assignment}\}$

$x * (-1 - x) < 0$

END -----

2. $\{\text{follows from A}\} 1 + x > 0$

3. $\{\text{follows from 2}\} -(1 + x) < 0$

4. $\{\text{follows from 3}\} -1 - x < 0$

6. $\{\text{follows from A,3}\} x * (-1 - x) < 0$

7. $\{\text{rewrite 6 with 1}\} \text{wp } (y := -1-x; z := x*y) (z < 0)$

END-----

Example 2

$\{^* 0 \leq i \wedge \neg \text{found} \wedge (\text{found} = (\exists k : 0 \leq k < i : a[k]=x)) \}^*$

$\text{found} := a[i]=x ;$

$(a[i]=x) = (\exists k : 0 \leq k < i+1 : a[k]=x)$

$i := i+1$

$\text{found} = (\exists k : 0 \leq k < i+1 : a[k]=x)$

$\{^* \text{found} = (\exists k : 0 \leq k < i : a[k]=x) \}^*$

$\text{wp } (x:=e) Q = Q[e/x]$

Example 2

Thm. Suppose $0 \leq i \wedge \neg \text{found} \wedge (\text{found} = (\exists k : 0 \leq k < i : a[k]=x))$.
Then $(a[i]=x) = (\exists k : 0 \leq k < i+1 : a[k]=x)$.

Proof. We have

$$\begin{aligned} (\exists k : 0 \leq k < i+1 : a[k]=x) &= (\exists k : 0 \leq k < i : a[k]=x) \vee a[i]=x && \text{(A.4.11.2)} \\ &= \text{found} \vee a[i]=x \\ &= \text{false} \vee a[i]=x \\ &= a[i]=x. \end{aligned}$$

Example 2

PROOF top

[A:] $0 \leq i \wedge \neg \text{found} \wedge (\text{found} = (\exists k : 0 \leq k < i : a[k]=x))$

[G:] $\text{wp}(\text{found} := a[i]=x ; i:=i+1) (\text{found} = (\exists k : 0 \leq k < i : a[k]=x))$

BEGIN-----

1. {sub1} $\text{wp}(\text{found}:=a[i]=x ; i:=i+1) (\text{found} = (\exists k:0 \leq k < i: a[k]=x)) = ((a[i]=x) = (\exists k:0 \leq k < i+1: a[k]=x))$

EQUATIONAL PROOF sub1 -----

$\text{wp}(\text{found} := a[i]=x ; i:=i+1) (\text{found} = (\exists k : 0 \leq k < i : a[k]=x))$

= {wp formula for sequence}

$\text{wp}(\text{found} := a[i]=x) (\text{wp}(i:=i+1) (\text{found} = (\exists k : 0 \leq k < i : a[k]=x)))$

= {wp formula for assignment}

$\text{wp}(\text{found} := a[i]=x) (\text{found} = (\exists k : 0 \leq k < i+1 : a[k]=x))$

= {wp formula for assignment}

$(a[i]=x) = (\exists k : 0 \leq k < i+1 : a[k]=x)$

END -----

2. { $\wedge$ elimination from A } $\neg \text{found}$

3. { $\wedge$ elimination from A } $\text{found} = (\exists k : 0 \leq k < i : a[k]=x)$

4. {rewriting 3 in 2} $\neg(\exists k : 0 \leq k < i : a[k]=x)$

5. { $\vee$ introduction } $(a[i]=x) \Rightarrow (\exists k : 0 \leq k < i : a[k]=x) \vee (a[i]=x)$

6. { $\vee$ elimination from 4 } $(\exists k : 0 \leq k < i : a[k]=x) \vee (a[i]=x) \Rightarrow (a[i]=x)$

7. {def of = from 5,6} $(a[i]=x) = (\exists k : 0 \leq k < i : a[k]=x) \vee (a[i]=x)$

8. {sub2} $(a[i]=x) = (\exists k : 0 \leq k < i+1 : a[k]=x)$

EQUATIONAL PROOF sub2 -----

$a[i]=x$

= {7}

$(\exists k : 0 \leq k < i : a[k]=x) \vee (a[i]=x)$

= {singleton domain}

$(\exists k : 0 \leq k < i : a[k]=x) \vee (\exists k : i \leq k < i+1 : a[k]=x)$

= {domain split}

$(\exists k : 0 \leq k < i+1 : a[k]=x)$

END-----

10. {rewrite 1 in 8} $\text{wp}(\text{found} := a[i]=x ; i:=i+1) (\text{found} = (\exists k : 0 \leq k < i : a[k]=x))$

END-----

Example 3

$\{^* \text{odd } x \wedge \text{odd } y \}^*$ $\Downarrow$



$((x > y+1) \wedge \text{odd } x) \vee ((x \leq y+1) \wedge \text{odd } (x+y+1))$

$y := y+1;$



$((x > y) \wedge \text{odd } x) \vee ((x \leq y) \wedge \text{odd } (x+y))$

if $x > y$ then $x := x - y$ else skip



$\text{odd } (x+y)$



$\{^* \text{odd } (x+y) \}^*$

$\text{odd } ((x - y) + y) = \text{odd } x$

$\text{wp } (x:=e) \ Q = Q[e/x]$
 $\text{wp } (\text{if } g \text{ then } S \text{ else } T) \ Q = (g \wedge \text{wp } S \ Q) \vee (\neg g \wedge \text{wp } T \ Q)$

Example 3

Let's check that

$$\text{odd } x \wedge \text{odd } y \Rightarrow ((x > y+1) \wedge \text{odd } x) \vee ((x \leq y+1) \wedge \text{odd } (x+y+1)).$$

We have

$$\begin{aligned} & \text{odd } x \wedge \text{odd } y \\ &= \quad \text{\textcolor{blue}{\{law of excluded middle\}}} \\ & ((x > y+1) \wedge (\text{odd } x \wedge \text{odd } y)) \vee ((x \leq y+1) \vee (\text{odd } x \wedge \text{odd } y)) \\ & \Rightarrow \quad \text{\textcolor{blue}{\{since odd } x \wedge \text{odd } y \text{ implies odd } x \text{ and odd } (x+y+1)\}}} \\ & ((x > y+1) \wedge (\text{odd } x)) \vee ((x \leq y+1) \wedge (\text{odd } (x+y+1))). \end{aligned}$$

Example 3

PROOF top

[A:] odd x $\wedge$ odd y

[G:] wp (y := y + 1; if x > y then x := x - y else skip) (odd (x + y))

BEGIN-----

1. {sub1} wp (y := y + 1; if x > y then x := x - y else skip) (odd (x + y)) = ((x > y + 1) $\wedge$ odd x) $\vee$ ((x $\leq$ y + 1) $\wedge$ odd (x + y + 1)).

EQUATIONAL PROOF sub1 -----

wp (y := y + 1; if x > y then x := x - y else skip) (odd (x + y))

= {wp formula for sequence}

wp (y := y + 1) (wp (if x > y then x := x - y else skip) (odd (x + y)))

= {wp formula for if-then-else}

wp (y := y + 1) (x > y $\wedge$ wp (x := x - y) (odd (x + y)) $\vee$ (x $\leq$ y $\wedge$ wp (skip) (odd (x + y))))

= {wp formula for assignment}

wp (y := y + 1) (x > y $\wedge$ odd (x - y + y)) $\vee$ ($\neg$ (x > y) $\wedge$ wp (skip) (odd (x + y)))

= {wp formula for skip}

wp (y := y + 1) (x > y $\wedge$ odd (x - y + y)) $\vee$ ($\neg$ (x > y) $\wedge$ odd (x + y))

= {wp formula for assignment}

x > y + 1 $\wedge$ odd (x - (y + 1) + (y + 1)) $\vee$ $\neg$ (x > (y + 1)) $\wedge$ odd (x + (y + 1))

= {trivial}

(x > y + 1 $\wedge$ odd x) $\vee$ (x $\leq$ y + 1) $\wedge$ odd (x + y + 1))

END-----

2. {law of excluded middle} (x > y + 1) $\vee$ (x $\leq$ y + 1)

3. { $\wedge$ elimination from A} odd x

4. { $\wedge$ elimination from A} odd y

5. {trivial from 3,4} odd (x + y + 1)

6. {sub2} (x > y + 1) $\Rightarrow$ (x > y + 1 $\wedge$ odd x) $\vee$ (x $\leq$ y + 1) $\wedge$ odd (x + y + 1))

PROOF sub2 -----

[A:] x > y + 1

[G:] (x > y + 1 $\wedge$ odd x) $\vee$ (x $\leq$ y + 1 $\wedge$ odd (x + y + 1))

BEGIN-----

1. { $\wedge$ introduction from A,3} x > y + 1 $\wedge$ odd x

2. { $\vee$ introduction from 1} (x > y + 1 $\wedge$ odd x) $\vee$ (x $\leq$ y + 1 $\wedge$ odd (x + y + 1))

END-----

7. {sub3} (x $\leq$ y + 1) $\Rightarrow$ (x > y + 1 $\wedge$ odd x) $\vee$ (x $\leq$ y + 1) $\wedge$ odd (x + y + 1))

PROOF sub3

[A:] x $\leq$ y + 1

[G:] (x > y + 1 $\wedge$ odd x) $\vee$ (x $\leq$ y + 1 $\wedge$ odd (x + y + 1))

BEGIN-----

1. { $\wedge$ introduction from A,5} (x $\leq$ y + 1) $\wedge$ odd (x + y + 1)

2. { $\vee$ introduction from 1} (x > y + 1 $\wedge$ odd x) $\vee$ (x $\leq$ y + 1) $\wedge$ odd (x + y + 1))

END-----

8. {case split on 2,6,7} (x > y + 1 $\wedge$ odd x) $\vee$ (x $\leq$ y + 1) $\wedge$ odd (x + y + 1))

9. {rewriting 1 in 8} wp (y := y + 1; if x > y then x := x - y else skip) (odd (x + y))

END-----

Some notes about the verification approach

- In our training, we prove $P \Rightarrow W$ by hand.
- In practice, to some degree this can be automatically checked using e.g. a SAT solver. With a SAT solver we instead check if:

$\neg(P \Rightarrow W)$ is **satisfiable**

If it is, then $P \Rightarrow W$ is **not** valid. Otherwise $P \Rightarrow W$ is valid.

- If the definition of wp is also “complete” (in addition to being “sound”), the witness of the satisfiability of $\neg(P \Rightarrow W)$ is essentially an input that will expose a bug in the program → useful for debugging.

Questions

- (a) $\{ * \mathbf{wp} \ S \ Q \ * \} \ S \ \{ * Q \ * \}$ is always a valid specification.
- (b) $\{ * P \Rightarrow (\mathbf{wp} \ S \ Q) \ * \} \ S \ \{ * Q \ * \}$ is always a valid specification.
- (c) $\{ * Q \ * \} \ S \ \{ * \mathbf{wp} \ S \ Q \ * \}$ is always a valid specification.
- (d) $\{ * P \ * \} \ S \ \{ * Q \ * \}$ is valid if and only if the predicate $\mathbf{wp} \ S \ (P \Rightarrow Q)$ is valid.

Questions

The specification $\{ * Q * \} S \{ * \text{true} * \}$ is known to be valid under **total correctness**. Which of the following conclusions is correct?

- (a) S will terminate when executed in any state.
- (b) If the implication $P \Rightarrow Q$ is valid, then S will terminate when executed in any state satisfying P .
- (c) The specification $\{ * Q * \} S \{ * Q \Rightarrow R * \}$ is also valid under partial correctness.
- (d) The specification $\{ * P \Rightarrow Q * \} S \{ * \text{true} * \}$ is also valid under total correctness.